

第3章：递归法

算法设计与分析



主讲教师：胡一涛

目录大纲

□ 递归法的基本概念

- 阶乘函数
- Fibonacci数列



□ 递归法的解题思路

- 递归和迭代
- 汉诺塔问题



□ 解递归的三种方法

• 递归树

(Recursion tree)

• 主方法

(Master method)

• 代入法

(Substitution method)



考察重点



熟练掌握技巧

目录大纲

□ 递归法的基本概念

- 阶乘函数
- Fibonacci数列



□ 递归法的解题思路

- 递归和迭代
- 汉诺塔问题



□ 解递归的三种方法

- 递归树
(Recursion tree)
- 主方法
(Master method)
- 代入法
(Substitution method)



考察重点



熟练掌握技巧

递归的概念

递归算法是直接或间接地调用自身的算法

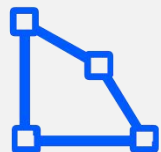
递归函数

递归
方程



用较小自变量的函数值来表示较大自变量的函数值

边界
条件



非递归定义的初始值

例1 阶乘函数

□ 通项公式为： $n! = 1 \cdot 2 \cdot 3 \cdots (n - 1) \cdot n$

□ 阶乘函数可递归地**定义**为：

$$n! = \begin{cases} 1 & n = 0 \\ n(n - 1)! & n > 0 \end{cases}$$

边界条件
递归方程

□ **递归方程**与**边界条件**是递归函数的两个要素。递归函数只有具备了这两个要素，才能在有限次计算后得出结果

例1 阶乘函数

- 阶乘函数可以递归地计算如下：

```
public static int factorial (int n)
{
    if (n==0) return 1;
    return n*factorial (n-1);
}
```

- 时间复杂度函数为：

$$T(n) = \begin{cases} 0 & n = 0 \\ T(n-1) + 1 & n > 0 \end{cases}$$



$$T(n) = \begin{cases} 0 & n = 0 \\ n & n > 0 \end{cases}$$

目录大纲

□ 递归法的基本概念

- 阶乘函数
- Fibonacci数列



□ 递归法的解题思路

- 递归和迭代
- 汉诺塔问题



□ 解递归的三种方法

- 递归树
(Recursion tree)
- 主方法
(Master method)
- 代入法
(Substitution method)



考察重点

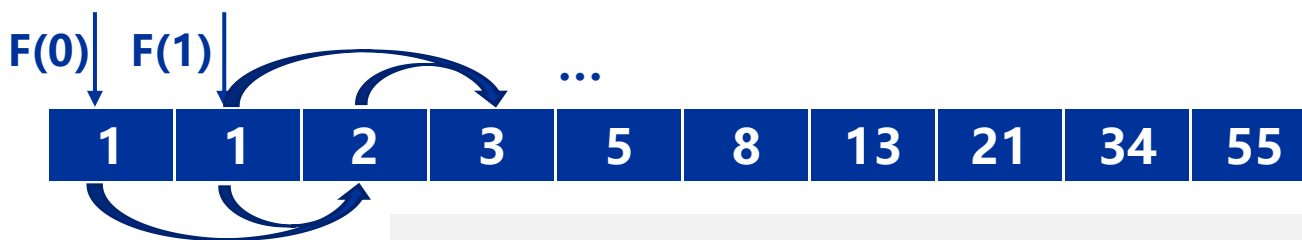


熟练掌握技巧

例2 Fibonacci数列的定义

- 无穷数列1, 1, 2, 3, 5, 8, 13, 21, 34, 55,, 称为Fibonacci数列, 可递归地定义为:

$$F(n) = \begin{cases} 1 & n = 0 \\ 1 & n = 1 \\ F(n-1) + F(n-2) & n > 1 \end{cases}$$



递归计算 ➡

```
int fibonacci(int n)
{
    if (n <= 1) return 1;
    return fibonacci(n-1)+fibonacci(n-2);
}
```


例2 Fibonacci数列

□ 时间复杂度函数为: $T(n)=T(n-1)+T(n-2)+1$ (a)

□ 观察易得: $T(n-1) > T(n-2)$ (b)

将式(b)代入(a):

$$\begin{cases} T(n)=T(n-1)+T(n-2) > 2T(n-2) \\ T(n)=T(n-1)+T(n-2) < 2T(n-1) \end{cases} \Rightarrow 2T(n-2) < T(n) < 2T(n-1)$$

归纳法易得

$$n > 2$$

$$T(n) \leq 2^n$$

□ 递归算法存在的问题:

- 递归在处理问题时要反复调用函数, 增大了空间和时间开销。
- 递归过程还可能造成大量的重复计算。

目录大纲

□ 递归法的基本概念

- 阶乘函数
- Fibonacci数列

□ 递归法的解题思路

- 递归和迭代
- 汉诺塔问题

□ 解递归的三种方法

- 递归树
(Recursion tree)
- 主方法
(Master method)
- 代入法
(Substitution method)



考察重点

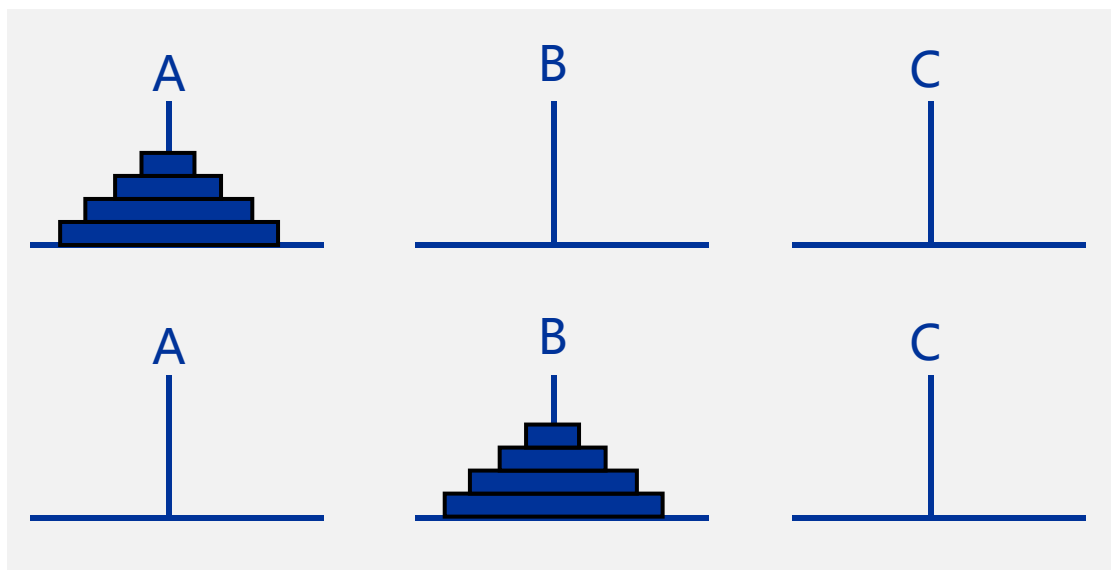


熟练掌握技巧

递归和迭代

- 效率和开销问题是递归最大的缺点
- 递归代码更加精炼，容易阅读
- 迭代算法不能求解的问题(如汉诺塔问题)，可以用递归的方法求解

汉诺塔问题



目录大纲

□ 递归法的基本概念

- 阶乘函数
- Fibonacci数列



□ 递归法的解题思路

- 递归和迭代
- 汉诺塔问题



□ 解递归的三种方法

- 递归树
(Recursion tree)
- 主方法
(Master method)
- 代入法
(Substitution method)



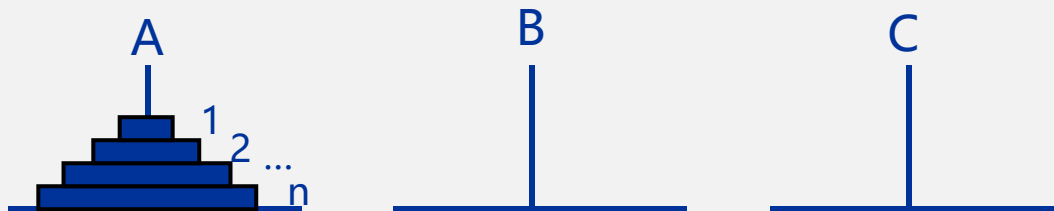
考察重点



熟练掌握技巧

汉诺塔问题：描述

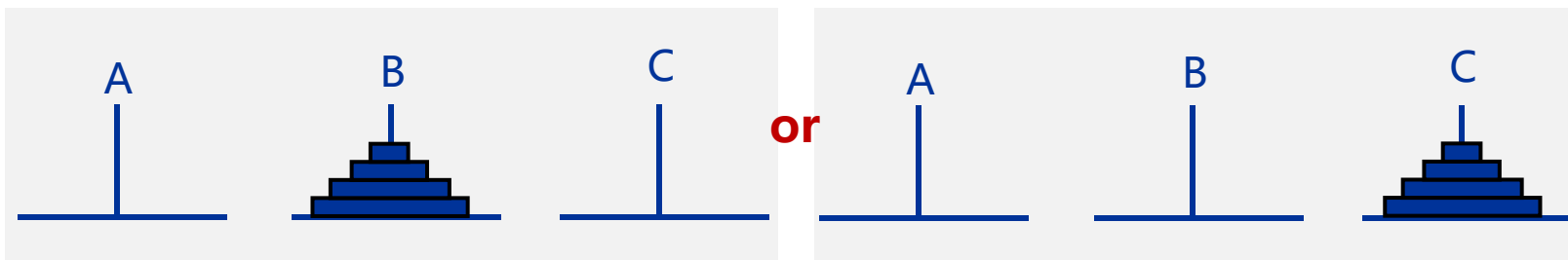
- 设 A, B, C 是 3 个塔座：塔座 A 上有 n 个圆盘，这些圆盘自下而上，由大到小地叠在一起，各圆盘从小到大编号为 $1, 2, \dots, n$



- **要求：**将塔座 A 上的圆盘移到塔座 B 上, 圆盘顺序不变.

- **移动规则**

1. 每次只能移动 1 个圆盘
2. 任何时刻都不允许将较大的圆盘压在较小的圆盘之上
3. 在满足移动规则 1 和 2 的前提下，可将圆盘移至 A, B, C 中任一塔座上

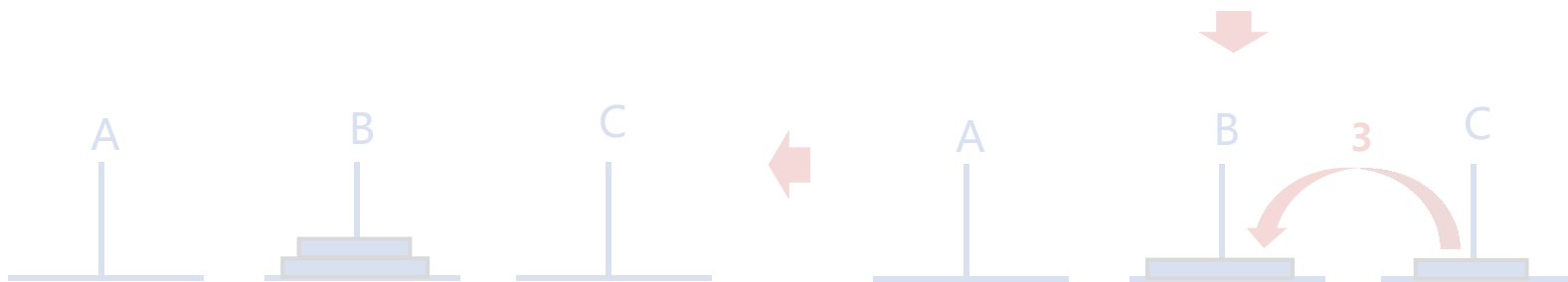


汉诺塔问题：实现

➤ $n=1$ 时，直接将圆盘从A挪到B上

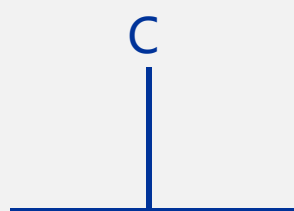
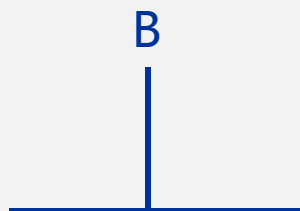
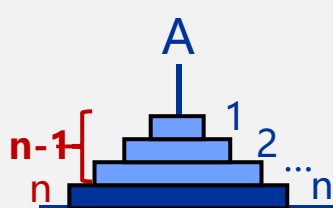


? $n > 2$ 时，该如何解决?

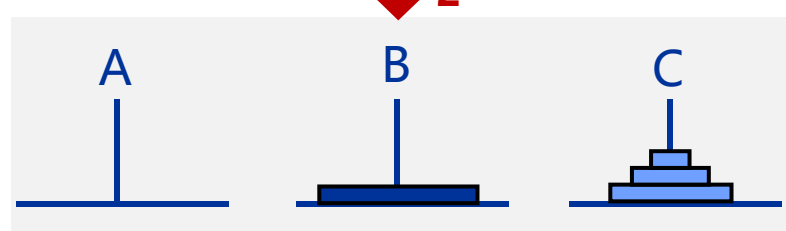
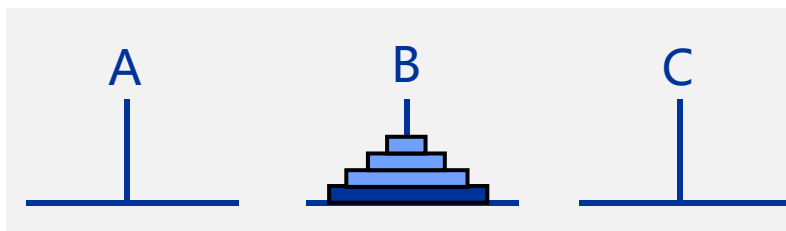
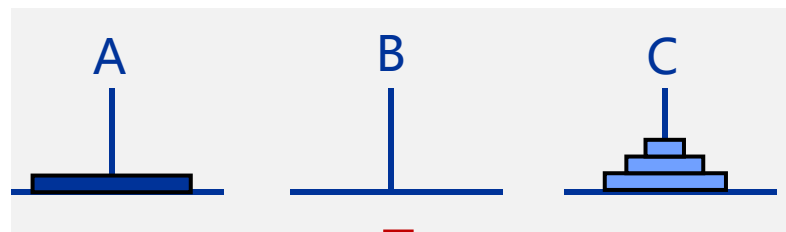
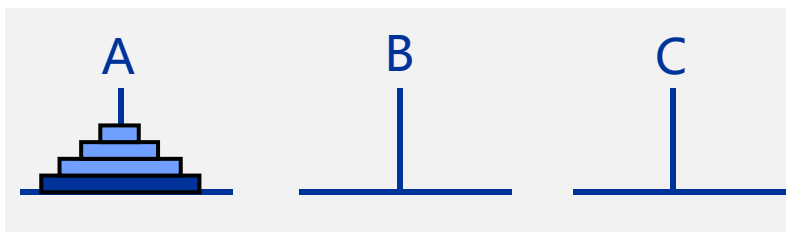


汉诺塔问题：实现

□ $n > 2$ 时，将 $n-1$ 个圆盘视作一个整体



简化成两层汉诺塔问题



汉诺塔问题：代码

□ 汉诺塔问题实现代码

```
void hanoi(int n, int a, int b, int c)
{
    if (n > 0)
    {
        hanoi(n-1, a, c, b); //a的n-1块当作整体挪到c上
        move(a,b); //a剩下的一块挪到b上
        hanoi(n-1, c, b, a); //c上的n-1块挪到b上
    }
}
```

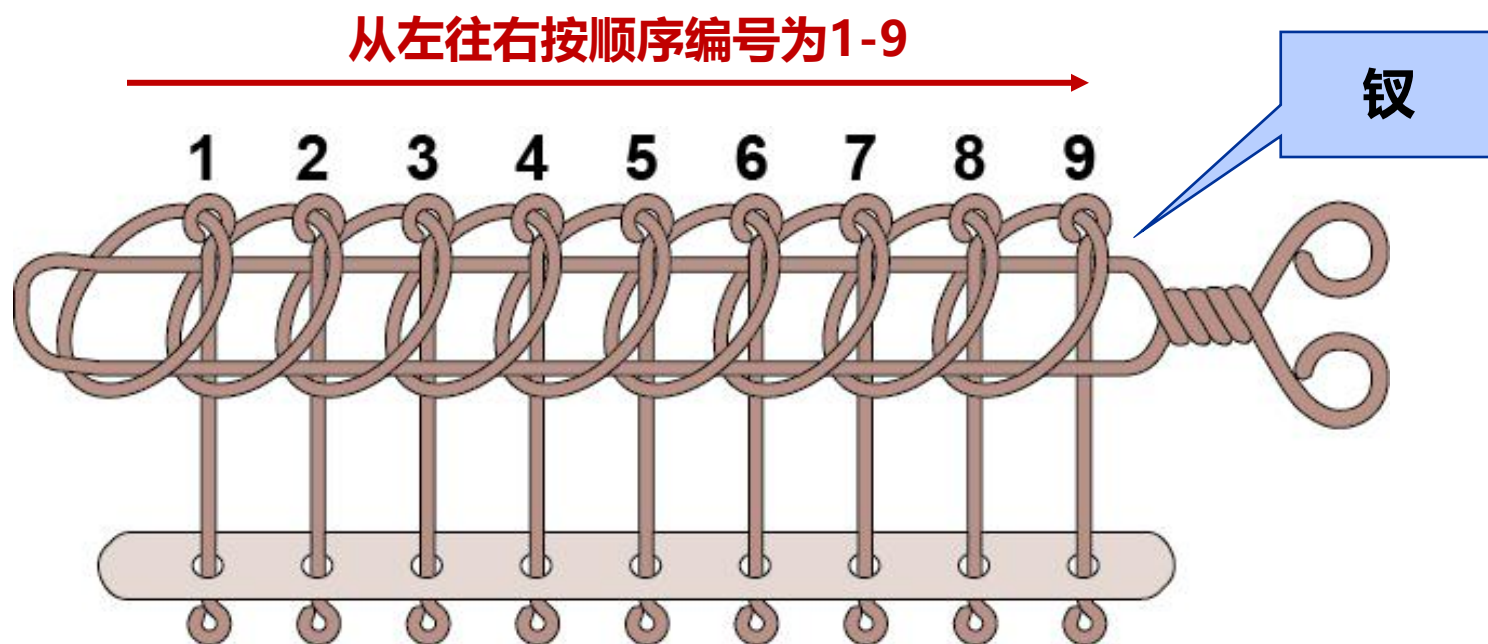
递归方程

$$T(n) = 2T(n - 1) + 1 = O(2^n)$$

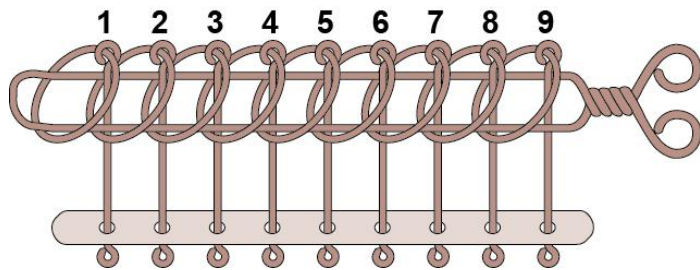
九连环问题（现场实验与讨论，5分钟 + 课间）

➤ 九连环是一种历史悠久的中国传统民间智力玩具

- 它以金属丝制成9个圆环，将圆环套装在横板或各式框架上，并贯以环柄
- 环柄部分称作“钗”；九个环**从左到右**依次称为**1到9号环**



九连环问题（现场实验与讨论，5分钟 + 课间）



九连环的三个基本动作

- 任何时候可以改变1号环的状态
 $b_9 b_8 b_7 \cdots b_2 0 \leftrightarrow b_9 b_8 b_7 \cdots b_2 1$
- 可以改变紧跟在领头环后的环的状态
 $b_9 \cdots b_i 0 1 0 0 \leftrightarrow b_9 \cdots b_i 1 1 0 0$
- 1号2号环状态相同时可以同时改变状态

取下 n-连环 的递归式解法

- $n=1$ ，直接用**操作1**取下1号环
- $n=2$ ，使用**操作2**下2号环后用**操作1**下1号环，或使用**操作3**一起下2号环和1号环
- $n>2$ ，则：
 - 使用解决**(n-2)-连环**的方法，将(n-2)号环至1号环全部取下
 - 使用**操作2**，取下n号环
 - **反向使用解决(n-2)-连环**的方法，将(n-2)号环至1号环全部装上
 - 使用解决**(n-1)-连环**的方法，将(n-1)号环至1号环全部取下

递归法设计的核心思想

1

假设函数 $f(n)$ 可以实现所需功能



2

可以将待解决的大问题分解成小问题

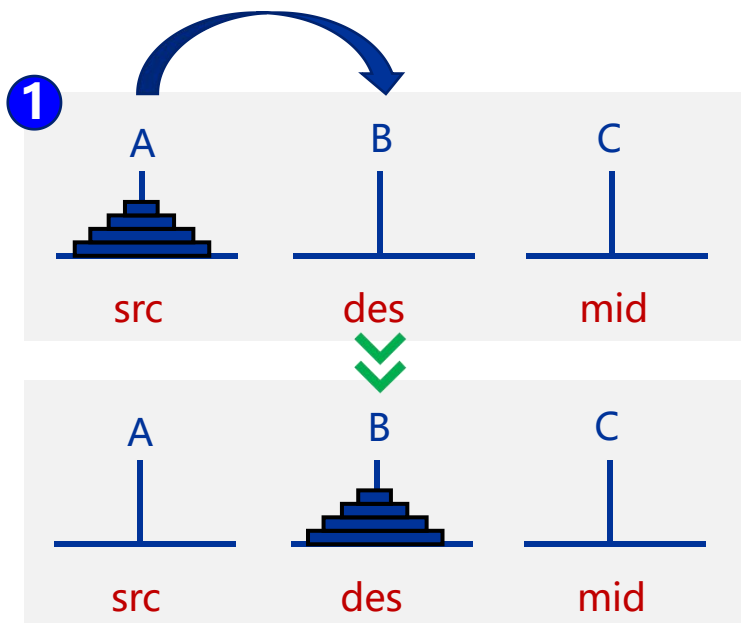


3

列出边界条件

汉诺塔问题

- ① 假设 $f(n,src,mid,des)$ 是实现将包含 n 个块的汉诺塔问题，从 src (源盘)，以 mid (中间盘)为媒介移动到 des (目标盘)上的函数



$f(n,src,mid,des)$

{

假设这个函数功能已经实现了
左侧问题的解就是：

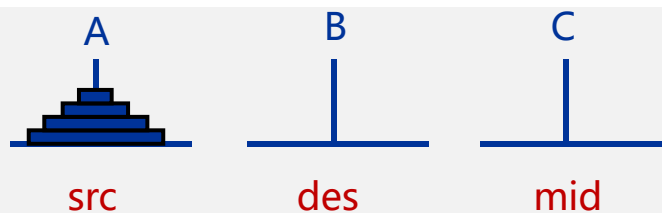
$f(4, A, C, B)$

Q: 如果只能用 $n < 4$ 的函数能
不能解决这个问题?

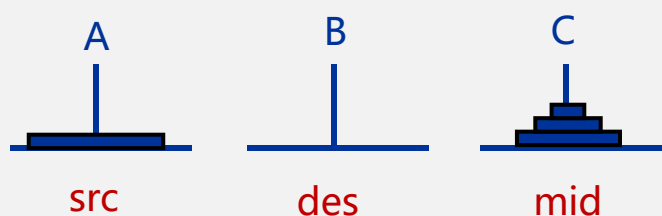
}

汉诺塔问题

1



2



```
f(n,src,mid,des)
```

```
{
```

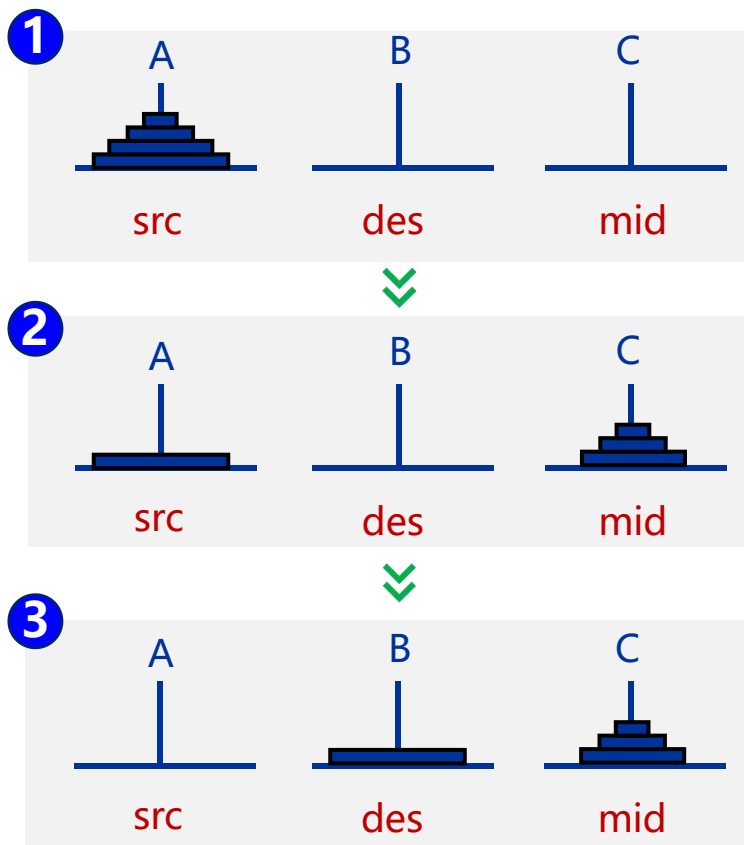
```
    f(n-1,A,B,C); //将n-1块视为  
    一个整体, 从A以B为媒介移动C上
```

```
    move(1,A,B); //将A盘上剩下的  
    第n块从A上移到B上
```

```
    f(n-1,C,A,B); //再调用f函数解  
    决C盘的n-1块汉诺塔问题, 以A为  
    媒介移动到B盘上
```

```
}
```

汉诺塔问题



f(n,src,mid,des)

{

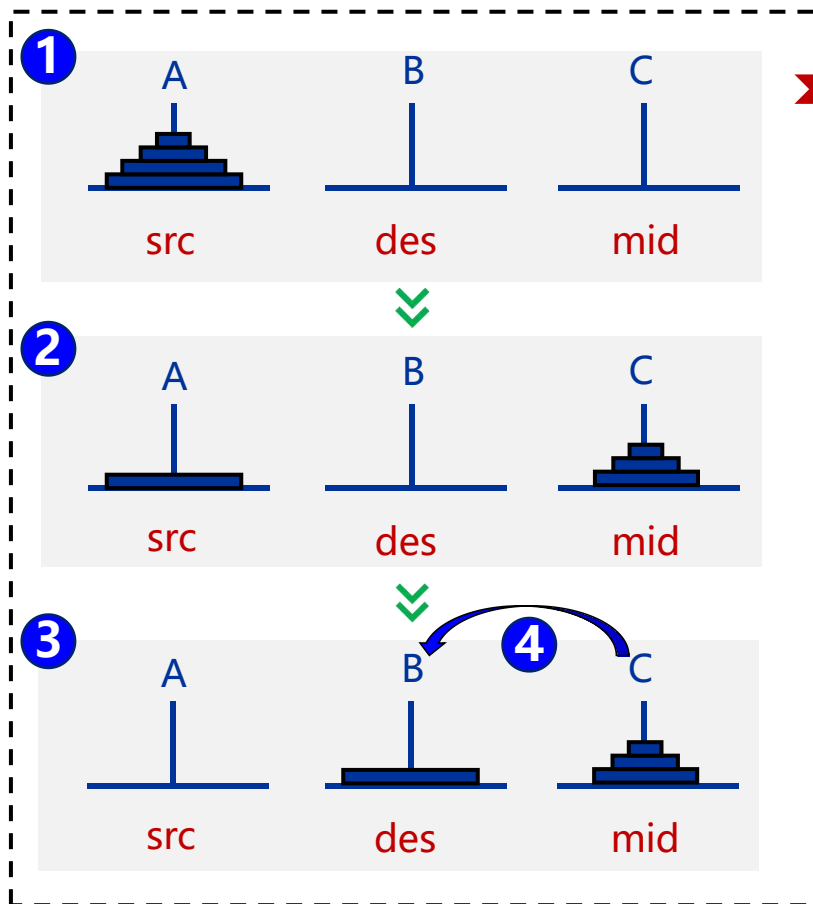
f(n-1,A,B,C);//将n-1块视为一个整体，从A以B为媒介移动C上

move(1,A,B);//将A盘上剩下的第n块从A上移到B上

f(n-1,C,A,B);//再调用f函数解决C盘的n-1块汉诺塔问题，以A为媒介移动到B盘上

}

汉诺塔问题



f(n,src,mid,des)

{

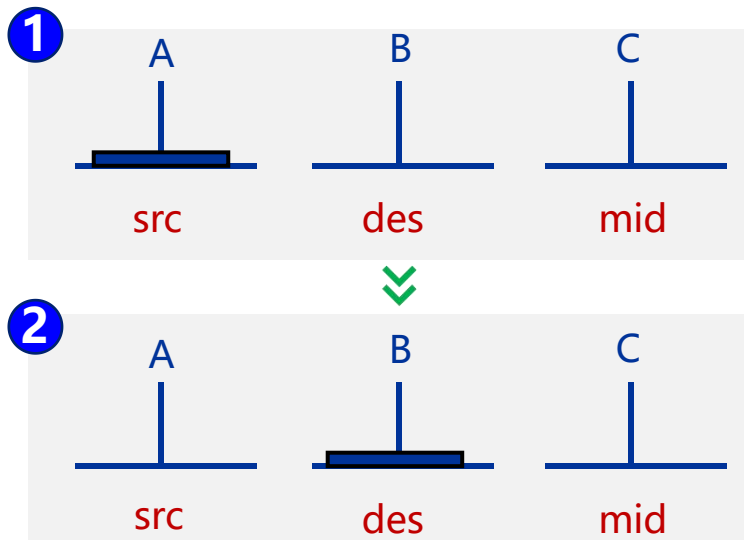
f(n-1,A,B,C);//将n-1块视为一个整体，从A以B为媒介移动C上

move(1,A,B);//将A盘上剩下的第n块从A上移到B上

f(n-1,C,A,B);//再调用f函数解决C盘的n-1块汉诺塔问题，以A为媒介移动到B盘上

}

汉诺塔问题：边界条件

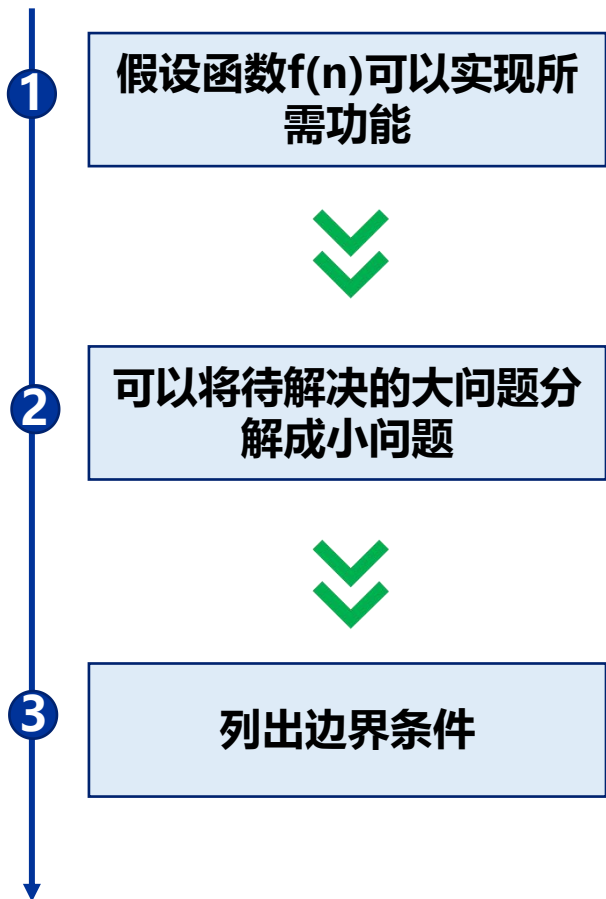


当递归到只剩一个板时，此时只需要通过一步就可以将板从A上移动到B上，这种情况为函数f的**边界条件**

f(n,src,mid,des)

```
{  
    f(n-1,A,B,C); //将n-1块视为  
    一个整体，从A以B为媒介移动C上  
    move(1,A,B); //将A盘上剩下的  
    第n块从A上移到B上  
    f(n-1,C,A,B); //再调用f函数解  
    决C盘的n-1块汉诺塔问题，以A为  
    媒介移动到B盘上  
}
```

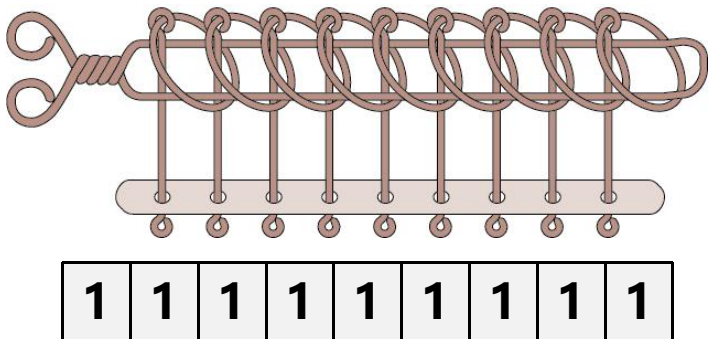

九连环



```
void DownRing(int n)
{
    if(n > 2) DownRing(n-2); // 如果环数大于2, 则调用DownRing(n-2)函数来卸下n-2环
    printf("下第%d环\n", n); // 卸下第n环
    if(n > 2) UpRing(n-2); // 如果环数大于2, 则装上n-2环
    if(n > 1) DownRing(n-1); // 卸下第n环后, 转化为卸n-1环的任务
}
```

九连环：卸除

① 假设DownRing(int n)是实现将n个九连环卸下的函数



void DownRing(int n)

{

假设这个函数功能已经实现了
左侧问题的解就是：

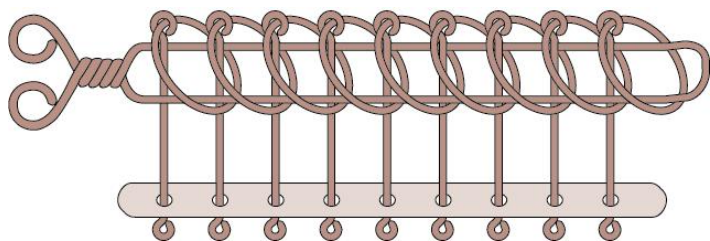
DownRing(9)

Q: 如果只能用 $n < 9$ 的函数不能解决这个问题?

}

九连环：卸除

① 假设DownRing(int n)是实现将n个九连环卸下的函数



1	1	1	1	1	1	1	1	1
1	1	0	0	0	0	0	0	0

```
void DownRing(int n)
```

```
{
```

```
    if(n > 2) DownRing(n-2); // 如果环数大于2, 则调用DownRing(n-2)函数来卸下n-2环
```

```
    printf("下第%d环\n", n); // 卸下第n环
```

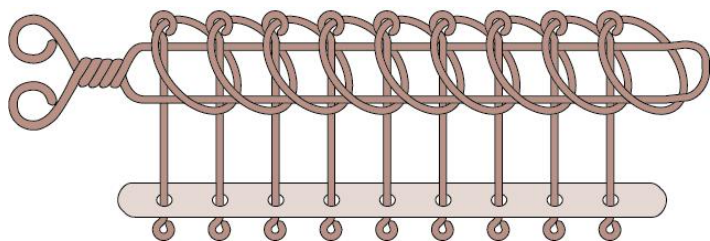
```
    if(n > 2) UpRing(n-2); // 如果环数大于2, 则装上n-2环
```

```
    if(n > 1) DownRing(n-1); // 卸下第n环后, 转化为卸n-1环的任务
```

```
}
```

九连环：卸除

① 假设DownRing(int n)是实现将n个九连环卸下的函数



1	1	1	1	1	1	1	1	1
1	1	0	0	0	0	0	0	0
0	1	0	0	0	0	0	0	0

void DownRing(int n)

{

if(n > 2) DownRing(n-2);//
 如果环数大于2, 则调用DownRing
 (n-2)函数来卸下n-2环

printf("下第%d环\n",n);//
 卸下第n环

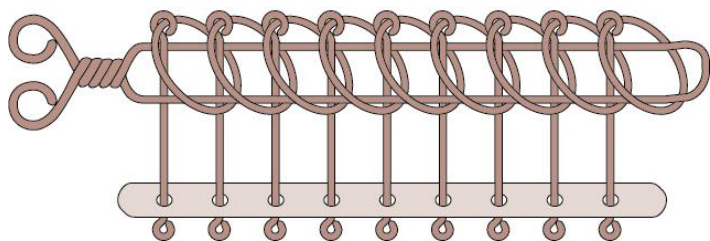
if(n > 2) UpRing(n-2);//如果
 环数大于2, 则装上n-2环

if(n > 1) DownRing(n-1);//卸
 下第n环后, 转化为卸n-1环的任务

}

九连环：卸除

① 假设DownRing(int n)是实现将n个九连环卸下的函数



1	1	1	1	1	1	1	1	1
1	1	0	0	0	0	0	0	0
0	1	0	0	0	0	0	0	0
0	1	1	1	1	1	1	1	1

```
void DownRing(int n)
```

```
{
```

```
    if(n > 2) DownRing(n-2); // 如果环数大于2, 则调用DownRing(n-2)函数来卸下n-2环
```

```
    printf("下第%d环\n", n); // 卸下第n环
```

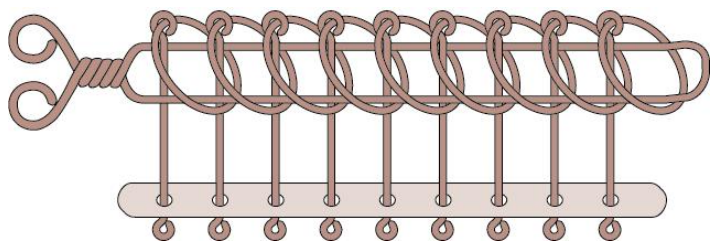
```
    if(n > 2) UpRing(n-2); // 如果环数大于2, 则装上n-2环
```

```
    if(n > 1) DownRing(n-1); // 卸下第n环后, 转化为卸n-1环的任务
```

```
}
```

九连环：卸除

① 假设DownRing(int n)是实现将n个九连环卸下的函数



1	1	1	1	1	1	1	1	1
1	1	0	0	0	0	0	0	0
0	1	0	0	0	0	0	0	0
0	1	1	1	1	1	1	1	1

八连环

```
void DownRing(int n)
```

```
{
```

```
    if(n>2) DownRing(n-2);//  
    如果环数大于2, 则调用DownRing  
    (n-2)函数来卸下n-2环
```

```
    printf("下第%d环\n",n);//  
    卸下第n环
```

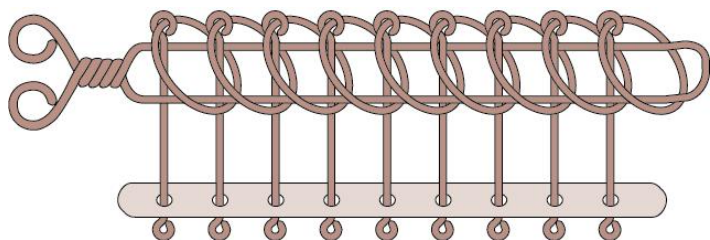
```
    if(n>2) UpRing(n-2);//如  
    果环数大于2, 则装上n-2环
```

```
    if(n>1) DownRing(n-1);//  
    卸下第n环后, 转化为卸n-1环的任  
    务
```

```
}
```

九连环：卸除边界条件

① 假设DownRing(int n)是实现将n个九连环卸下的函数



0	0	0	0	0	0	0	0	1
---	---	---	---	---	---	---	---	---

0	0	0	0	0	0	0	1	1
---	---	---	---	---	---	---	---	---

0	0	0	0	0	0	0	0	0
---	---	---	---	---	---	---	---	---

当只剩下第1环或者只剩下1、2环的时候，可以直接卸下

```
void DownRing(int n)
```

```
{
```

```
    if(n > 2) DownRing(n-2); // 如果环数大于2，则调用DownRing(n-2)函数来卸下n-2环
```

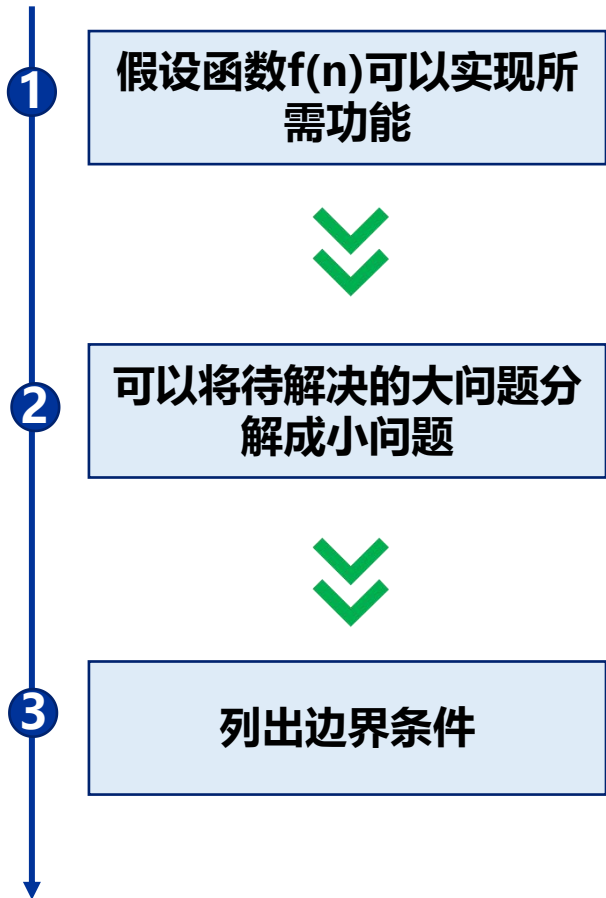
```
    printf("下第%d环\n", n); // 卸下第n环
```

```
    if(n > 2) UpRing(n-2); // 如果环数大于2，则装上n-2环
```

```
    if(n > 1) DownRing(n-1); // 卸下第n环后，转化为卸n-1环的任务
```

```
}
```

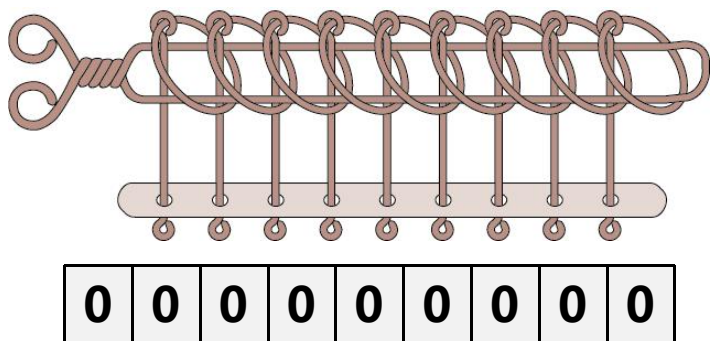
九连环：安上



```
void UpRing(int n)
{
    if(n>1) UpRing(n-1); //调用
    UpRing函数将n-1环安上
    if(n>2) DownRing(n-2); //再
    将n-2环卸下
    printf("上第%d环\n",n); //此
    时只有第n-1环在上面，可以直接上
    上第n环
    if(n>2) UpRing(n-2); //继续
    调用UpRing函数，解决n-2连环的
    安上问题
}
```


九连环：安上

① 假设UpRing(int n)是实现将n个九连环安上的函数



void UpRing(int n)

{

 if(n>1) UpRing(n-1); //调用UpRing函数将n-1环安上

 if(n>2) DownRing(n-2); //再将n-2环卸下

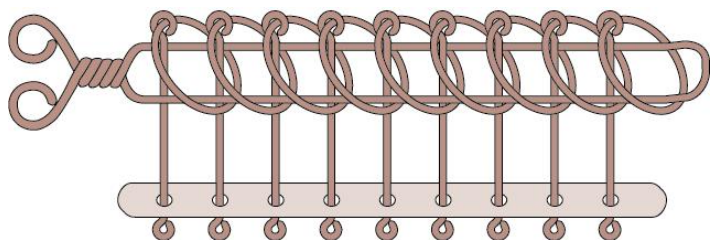
 printf("上第%d环\n",n); //此时只有第n-1环在上面，可以直接上上第n环

 if(n>2) UpRing(n-2); //继续调用UpRing函数，解决n-2连环的安上问题

}

九连环：安上

① 假设UpRing(int n)是实现将n个九连环安上的函数



0	0	0	0	0	0	0	0	0
0	1	1	1	1	1	1	1	1

```
void UpRing(int n)
```

```
{
```

```
    if(n > 1) UpRing(n-1); //调用UpRing函数将n-1环安上
```

```
    if(n > 2) DownRing(n-2); //再将n-2环卸下
```

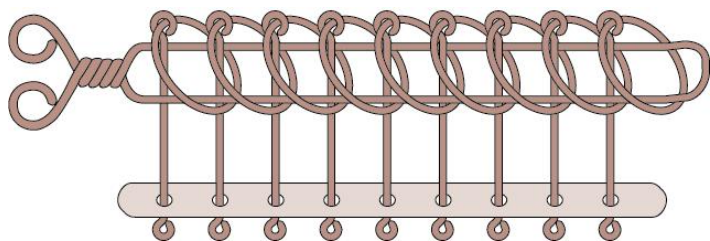
```
    printf("上第%d环\n", n); //此时只有第n-1环在上面，可以直接上上第n环
```

```
    if(n > 2) UpRing(n-2); //继续调用UpRing函数，解决n-2连环的安上问题
```

```
}
```

九连环：安上

① 假设UpRing(int n)是实现将n个九连环安上的函数



0	0	0	0	0	0	0	0	0
---	---	---	---	---	---	---	---	---

0	1	1	1	1	1	1	1	1
---	---	---	---	---	---	---	---	---

0	1	0	0	0	0	0	0	0
---	---	---	---	---	---	---	---	---

```
void UpRing(int n)
```

```
{
```

```
    if(n>1) UpRing(n-1); //调用UpRing函数将n-1环安上
```

```
    if(n>2) DownRing(n-2); //再将n-2环卸下
```

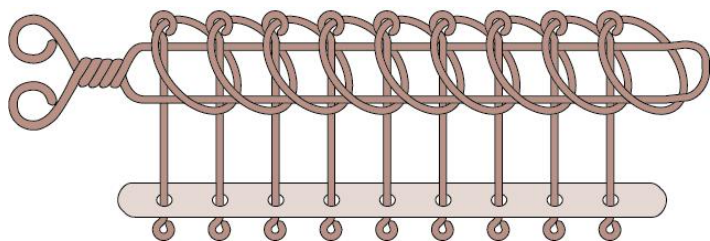
```
    printf("上第%d环\n",n); //此时只有第n-1环在上面，可以直接上上第n环
```

```
    if(n>2) UpRing(n-2); //继续调用UpRing函数，解决n-2连环的安上问题
```

```
}
```

九连环：安上

① 假设UpRing(int n)是实现将n个九连环安上的函数



0	0	0	0	0	0	0	0	0
0	1	1	1	1	1	1	1	1
0	1	0	0	0	0	0	0	0
1	1	0	0	0	0	0	0	0

```
void UpRing(int n)
```

```
{
```

```
    if(n>1) UpRing(n-1); //调用UpRing函数将n-1环安上
```

```
    if(n>2) DownRing(n-2); //再将n-2环卸下
```

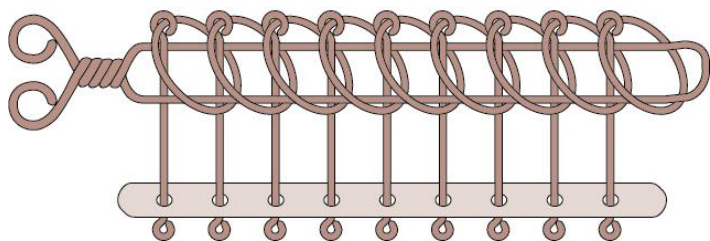
```
    printf("上第%d环\n",n); //此时只有第n-1环在上面，可以直接上上第n环
```

```
    if(n>2) UpRing(n-2); //继续调用UpRing函数，解决n-2连环的安上问题
```

```
}
```

九连环：安上

① 假设UpRing(int n)是实现将n个九连环安上的函数



0	0	0	0	0	0	0	0	0
0	1	1	1	1	1	1	1	1
0	1	0	0	0	0	0	0	0
1	1	0	0	0	0	0	0	0

七连环

```
void UpRing(int n)
```

```
{
```

```
    if(n > 1) UpRing(n-1); //调用UpRing函数将n-1环安上
```

```
    if(n > 2) DownRing(n-2); //再将n-2环卸下
```

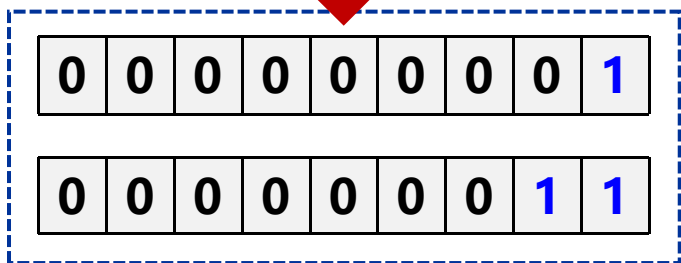
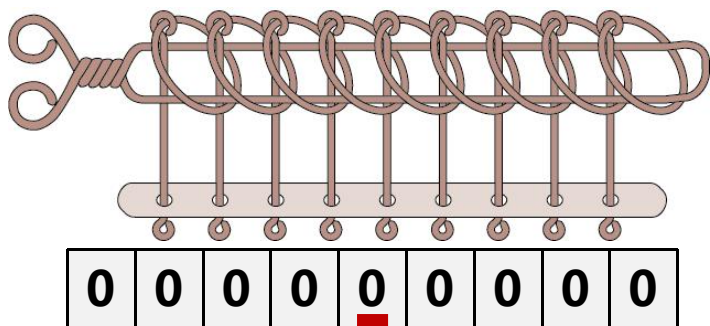
```
    printf("上第%d环\n", n); //此时只有第n-1环在上面，可以直接上上第n环
```

```
    if(n > 2) UpRing(n-2); //继续调用UpRing函数，解决n-2连环的安上问题
```

```
}
```

九连环：安上边界条件

① 假设UpRing(int n)是实现将n个九连环安上的函数



当初始没有环在的时候，可以直接装上第1环或者1、2环

```
void UpRing(int n)
```

```
{
```

```
    if(n>1) UpRing(n-1); //调用UpRing函数将n-1环安上
```

```
    if(n>2) DownRing(n-2); //再将n-2环卸下
```

```
    printf("上第%d环\n",n); //此时只有第n-1环在上面，可以直接上上第n环
```

```
    if(n>2) UpRing(n-2); //继续调用UpRing函数，解决n-2连环的安上问题
```

```
}
```

目录大纲

□ 递归法的基本概念

- 阶乘函数
- Fibonacci数列



□ 递归法的解题思路

- 递归和迭代
- 汉诺塔问题



□ 解递归的三种方法

• 递归树

(Recursion tree)

• 主方法

(Master method)

• 代入法

(Substitution method)



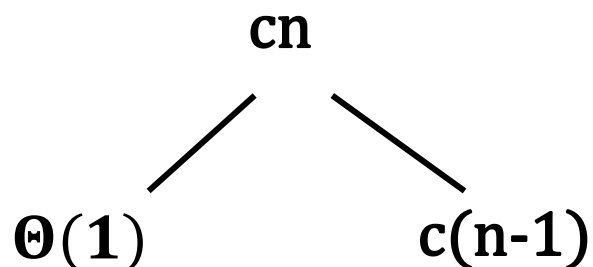
考察重点



熟练掌握技巧

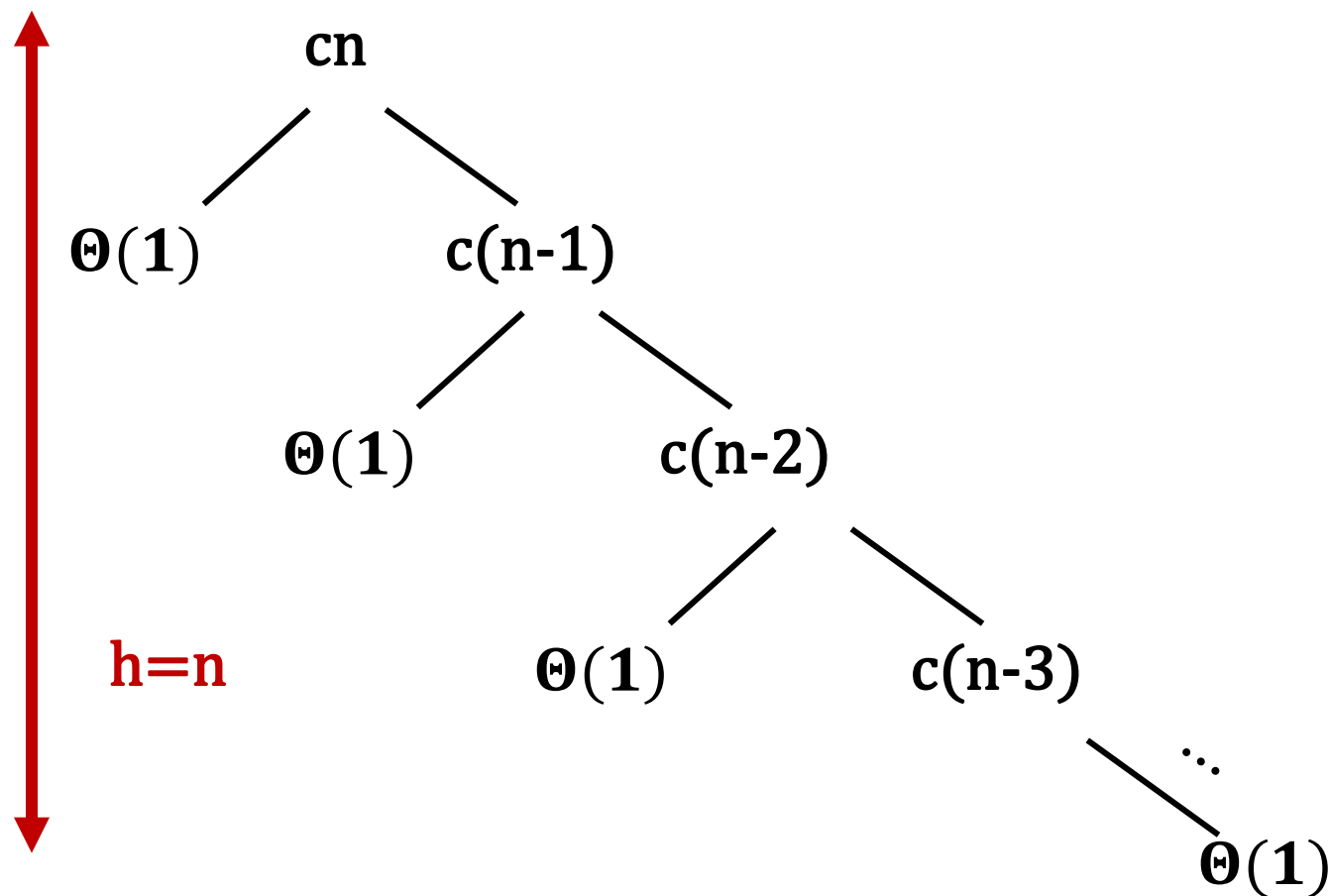
展开递归树（减法式递归）

□ 展开递归树 $T(n) = T(1) + T(n-1) + cn$ 并做渐进分析



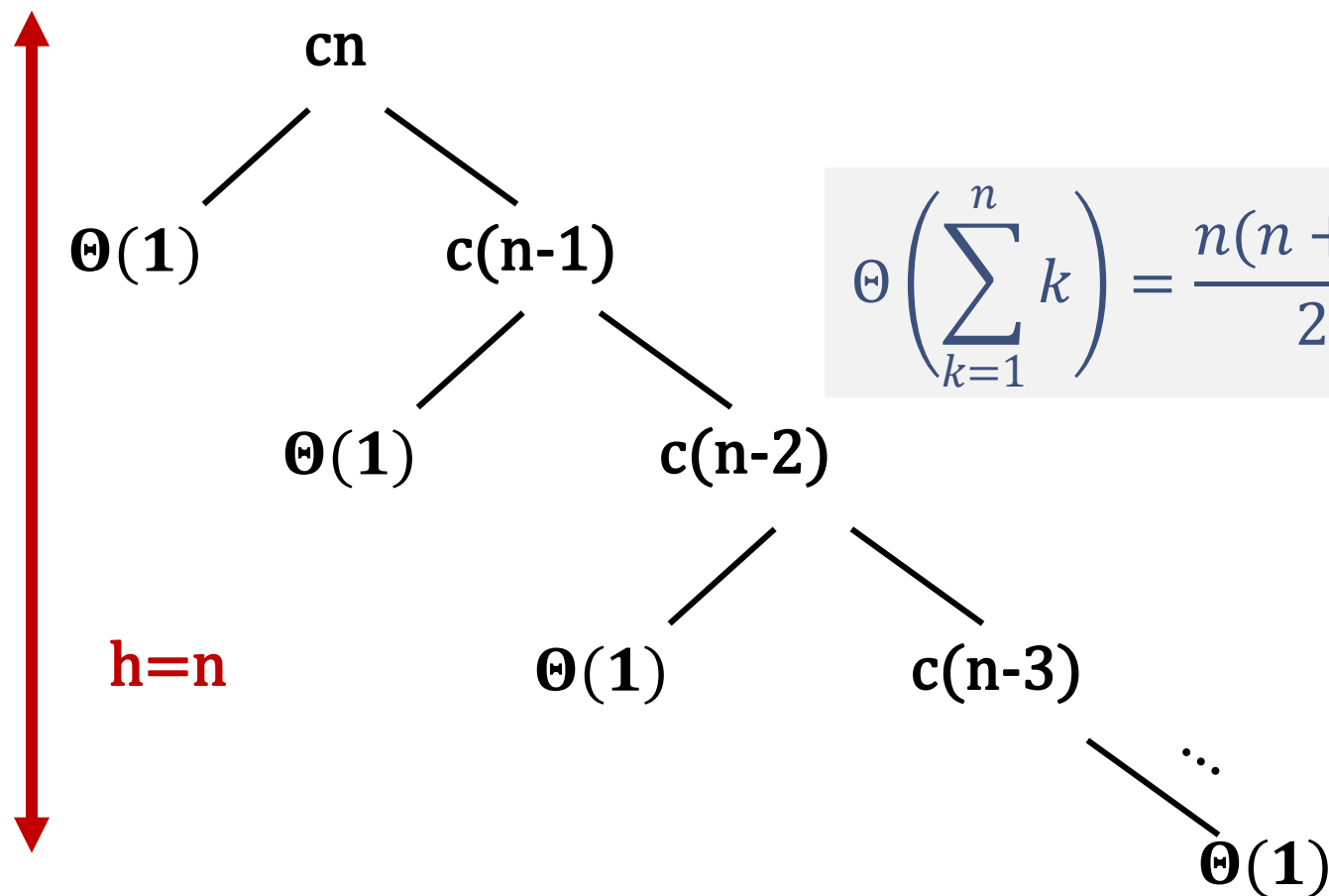
展开递归树（减法式递归）

□ 展开递归树 $T(n) = T(1) + T(n-1) + cn$ 并做渐进分析



展开递归树（减法式递归）

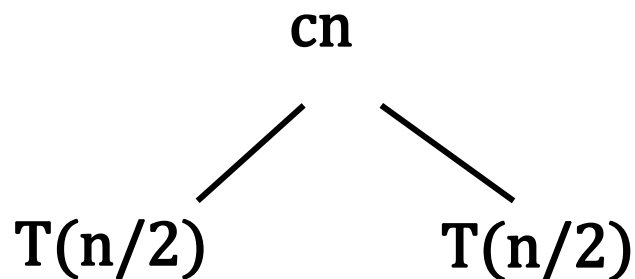
□ 展开递归树 $T(n) = T(1) + T(n-1) + cn$ 并做渐进分析



$$\Theta\left(\sum_{k=1}^n k\right) = \frac{n(n+1)}{2} = \Theta(n^2)$$

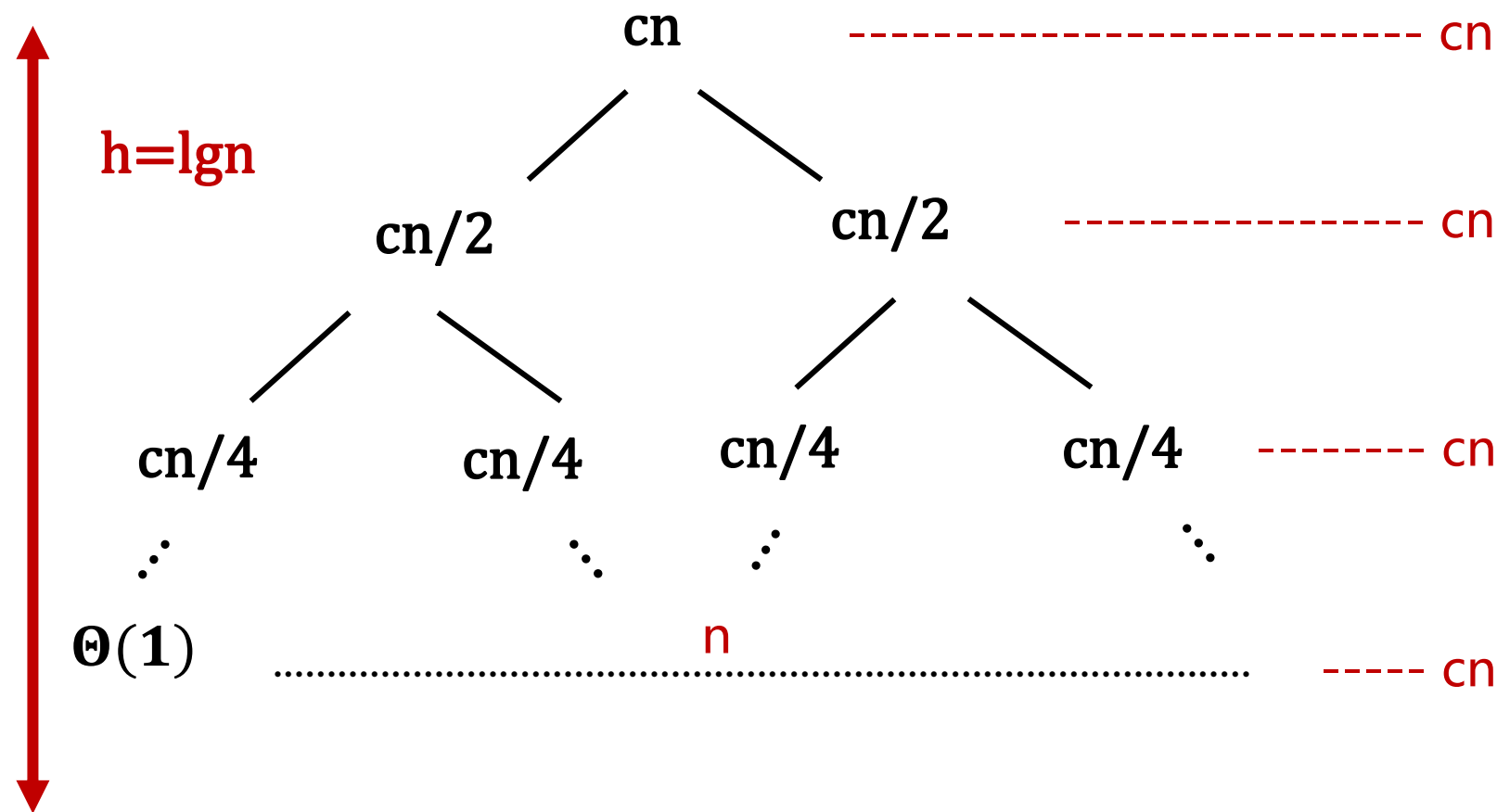
展开递归树（除法式递归）

□ 解 $T(n) = 2T(n/2) + cn$, 其中 $c > 0$, 为常数.



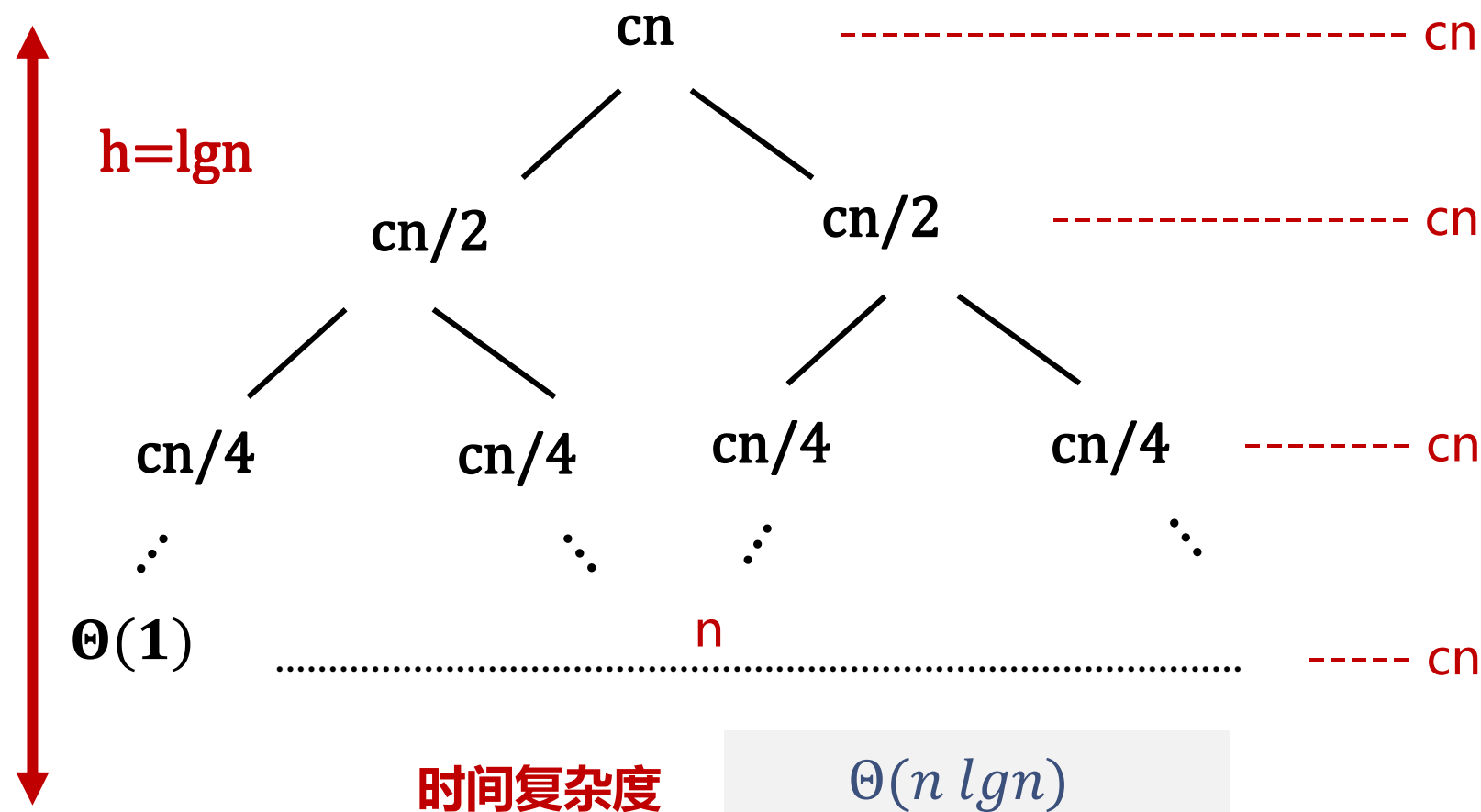
展开递归树 (除法式递归)

□ 解 $T(n) = 2T(n/2) + cn$, 其中 $c > 0$, 为常数.



展开递归树 (除法式递归)

□ 解 $T(n) = 2T(n/2) + cn$, 其中 $c > 0$, 为常数.



递归树的渐进值和深度概念

□ 计算 $T(n) = 2T(n/2) + cn$ 的渐近值和递归树的深度

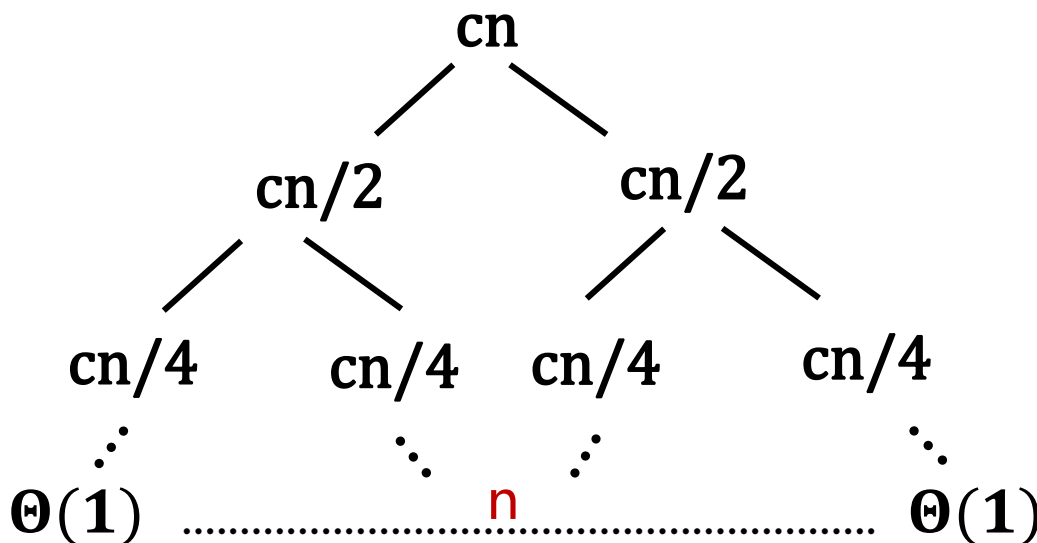
$$\begin{aligned} T(n) &= 2T(n/2) + cn \\ &= cn + 2 * cn/2 + 4 * cn/4 + \dots + n * c \\ &= cn * \lg n \\ &= \Theta(n \lg n) \end{aligned}$$

渐近值



深度

$$h = \lg n$$



递归式的一般性总结

□ 递归方程 $T(n)=aT(n/b)+f(n)$

参数的具体含义：

a：递归调用的数量；

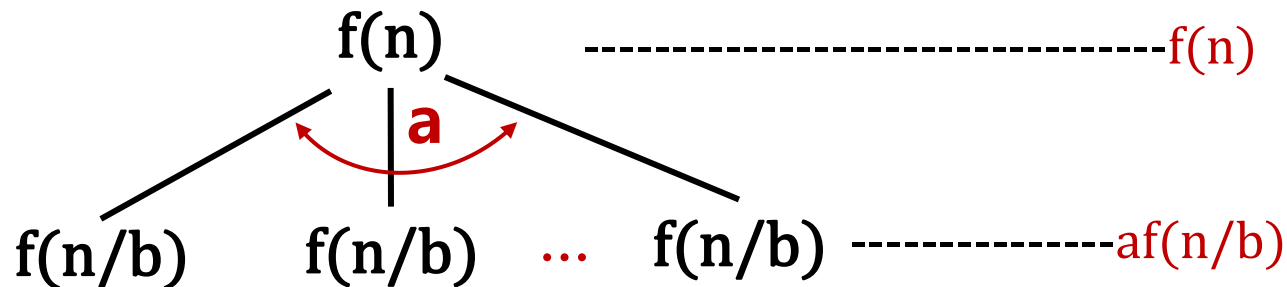
b：输入长度的收缩因子；

f(n)：“分解和组合步骤”的运行时间指数。

有 **a 个递归调用**，每个递归调用处理**1/b规模的数组**，并在递归调用之外完成 a 个数组的合并，代价为 f(n)

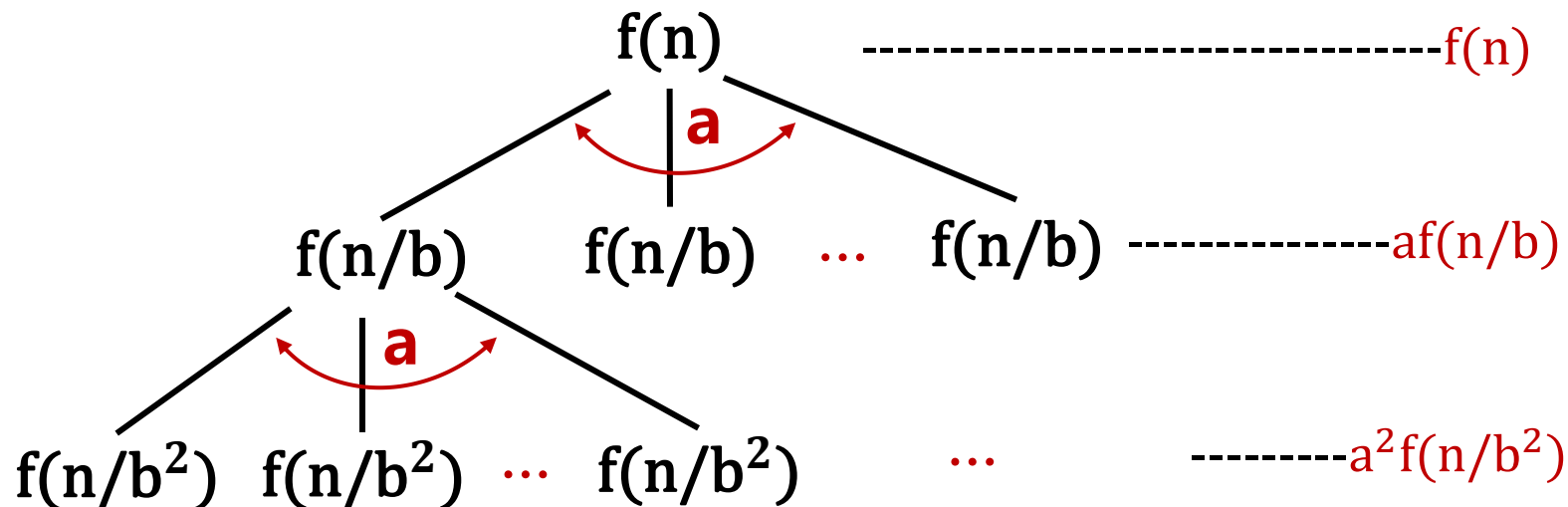
递归树的一般性总结：举例

□ 递归方程 $T(n) = aT(n/b) + f(n)$



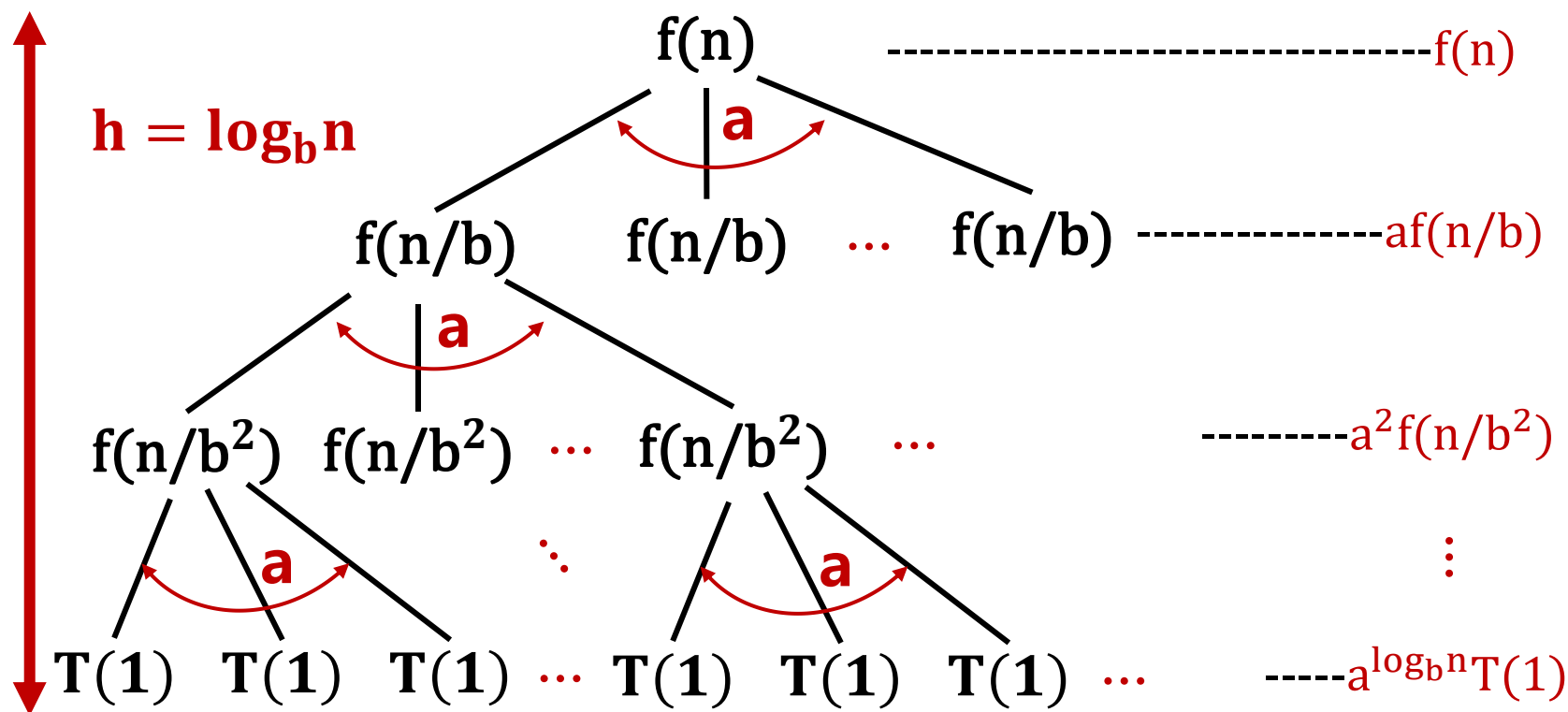
递归树的一般性总结：举例

□ 递归方程 $T(n) = aT(n/b) + f(n)$



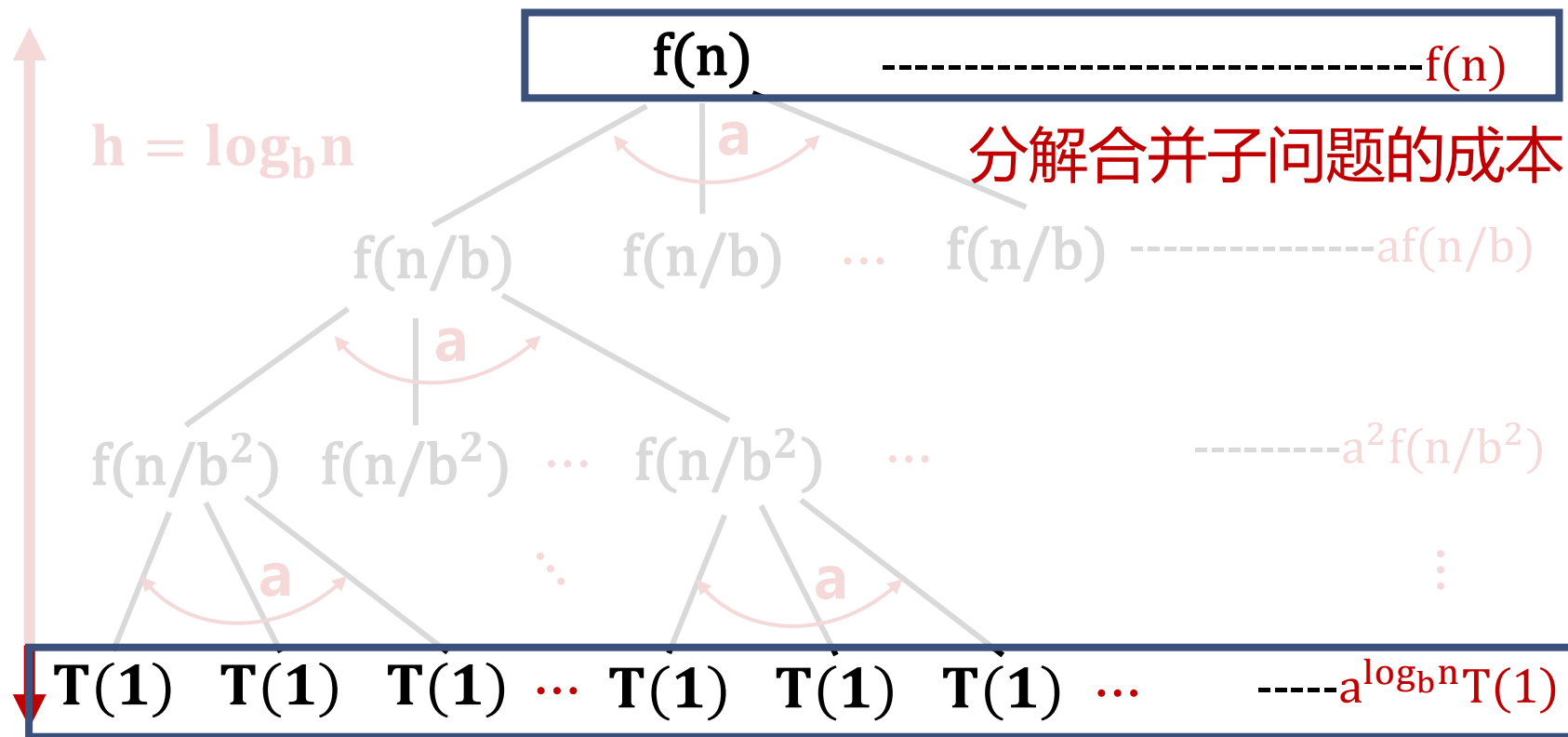
递归树的一般性总结：举例

□ 递归方程 $T(n) = aT(n/b) + f(n)$



递归树的一般性总结：举例

□ 递归方程 $T(n) = aT(n/b) + f(n)$



递归式

□ 从递归树可知:

$$T(n) = a^h T(1) + \sum_{k=0}^{h-1} a^k f\left(\frac{n}{b^k}\right) = a^{\log_b n} T(1) + \sum_{k=0}^{h-1} a^k f(b^{h-k})$$

$(h = \log_b a)$

最底层

解决子问题的成本

分解和合并

子问题的成本

□ **结论:**

$T(n)$ 的渐近阶由 $\sum_{k=0}^{h-1} a^k f(b^{h-k})$ 和 $a^{\log_b n}$ 中**阶较高**的决定!

递归式

□ 从递归树可知:

$$T(n) = a^h T(1) + \sum_{k=0}^{h-1} a^k f\left(\frac{n}{b^k}\right) = a^{\log_b n} T(1) + \sum_{k=0}^{h-1} a^k f(b^{h-k})$$

($h = \log_b a$)

$$a^{\log_b n}$$

换底公式

$$n^{\log_b a}$$

$$\sum_{j=0}^{k-1} a^j f(n/b^j) \leq \sum_{j=0}^{k-1} c^j f(n) \leq f(n) \sum_{j=0}^{\infty} c^j = \frac{f(n)}{1-c}$$

□ 结论:

$$(0 < c < 1)$$

$T(n)$ 的渐近阶由 $f(n)$ 和 $n^{\log_b a}$ 中阶较高的决定!

目录大纲

□ 递归法的基本概念

- 阶乘函数
- Fibonacci数列



□ 递归法的解题思路

- 递归和迭代
- 汉诺塔问题



□ 解递归的三种方法

- 递归树
(Recursion tree)
- **主方法**
(Master method)
- 代入法
(Substitution method)



考察重点



熟练掌握技巧

主方法

□ 主方法用来解下述递归

$$T(n) = aT(n/b) + f(n)$$

其中 $a \geq 1$, $b > 1$ 为整数, $f(n) > 0$.

按 $f(n)$ 相对于 $n^{\log_b a}$ 的渐近性质, 分三种情形进行分析.

主方法：情形1

□ 情形1:

- $f(n) = O(n^{\log_b a - \varepsilon})$, $\varepsilon > 0$ 为某一常数,
- $f(n)$ 的增长渐近地慢于 $n^{\log_b a}$ (慢 n^ε 倍).

□ 结论: $T(n) = \Theta(n^{\log_b a})$.

主方法： case1

$$\square T(n) = 9T(n/3) + \Theta(n)$$

- $a = 9, b = 3$
- $n^{\log_b a} = n^2$
- $f(n) = cn$
- $f(n) = O(n^{\log_b a - \varepsilon})$
- 其中 $\varepsilon = 1$,
- 满足情况1
- $T(n) = \Theta(n^2)$

$$\square T(n) = 4T(n/2) + n$$

- $a = 4, b = 2$
- $n^{\log_b a} = n^2$
- $f(n) = n$
- $f(n) = O(n^{2 - \varepsilon})$
- 其中 $\varepsilon = 1$,
- 满足情况1
- $T(n) = \Theta(n^2)$

主方法：情形2

□ 情形2:

- $f(n) = \Theta(n^{\log_b a})$, $k \geq 0$ 为某一常数.
- $f(n)$ 和 $n^{\log_b a}$ 几乎有相同的渐近增长率.

□ 结论: $T(n) = \Theta(n^{\log_b a} \lg n)$.

主方法：case2

□ $T(n) = 2T(n/2) + \Theta(n)$.

- $a = 2, b = 2$
- $n^{\log_b a} = n$
- $f(n) = cn$
- 满足情况 2
- $T(n) = \Theta(n \lg n)$

主方法：情形3

□ 情形3:

- $f(n) = \Omega(n^{\log_a + \varepsilon})$, $\varepsilon > 0$ 为一常数.
- $f(n)$ 多项式地快于 $n^{\log_b a}$ (by an n^ε factor),

同时, $f(n)$ 需要满足以下正则条件:

$$af(n/b) \leq cf(n), \quad 0 < c < 1 \text{ 为常数.}$$

□ 结论: $T(n) = \Theta(f(n))$.

主方法：case3

□ $T(n) = 3T(n/4) + n \lg n.$

- $a = 3, b = 4, f(n) = n \lg n, n^{\log_b a} = n^{\log_4 3} < n^{0.793}.$
- $f(n) = n \lg n > n > n^{\log_b a + 0.2},$ 其中 $\varepsilon = 0.2.$
- 同时, 当 n 足够大时, 对于 $c = 3/4,$ 有

$$af\left(\frac{n}{b}\right) = 3 \cdot \frac{n}{4} \lg \frac{n}{4} \leq \frac{3}{4} n \lg n = cf(n)$$

- 正则条件成立,
- 满足情况3
- 递归式的解为 $T(n) = \Theta(n \lg n)$

主方法：case3

$$\square T(n) = 4T(n/2) + n^3$$

- $a = 4, b = 2, f(n) = n^3, n^{\log_b a} = n^2$ 。
- $f(n) = n^3 > n^{\log_b a + 0.5}$, 其中 $\varepsilon = 0.5$ 。
- 同时, 当 n 足够大时, 对于 $c = 1/2$,

$$af\left(\frac{n}{b}\right) = 4 \cdot \left(\frac{n}{2}\right)^3 \leq \frac{1}{2}n^3 = cf(n)$$

- 正则条件成立,
- 满足情况3
- 递归式的解为 $T(n) = \Theta(n^3)$.

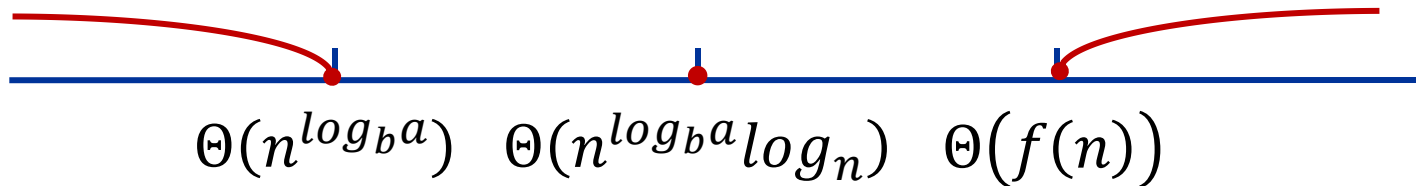
主方法：总结

□ 主定理：对形如 $T(n) = aT\left(\frac{n}{b}\right) + f(n)$ 的递归式

$$T(n) = \begin{cases} \Theta(f(n)) & \text{if } f(n) = \Omega(n^{\log_b a + \epsilon}) \quad \textcircled{1} \\ \Theta(n^{\log_b a} \log n) & \text{if } f(n) = \Omega(n^{\log_b a}) \quad \textcircled{2} \\ \Theta(n^{\log_b a}) & \text{if } f(n) = \Omega(n^{\log_b a - \epsilon}) \quad \textcircled{3} \end{cases}$$



$$\textcircled{1} \quad T(n) = \Theta(n^{\log_b a}) \quad \textcircled{2} \quad T(n) = \Theta(n^{\log_b a} \log n) \quad \textcircled{3} \quad T(n) = \Theta(f(n))$$



目录大纲

□ 递归法的基本概念

- 阶乘函数
- Fibonacci数列



□ 递归法的解题思路

- 递归和迭代
- 汉诺塔问题



□ 解递归的三种方法

- 递归树
(Recursion tree)
- 主方法
(Master method)
- **代入法**
(Substitution method)



考察重点

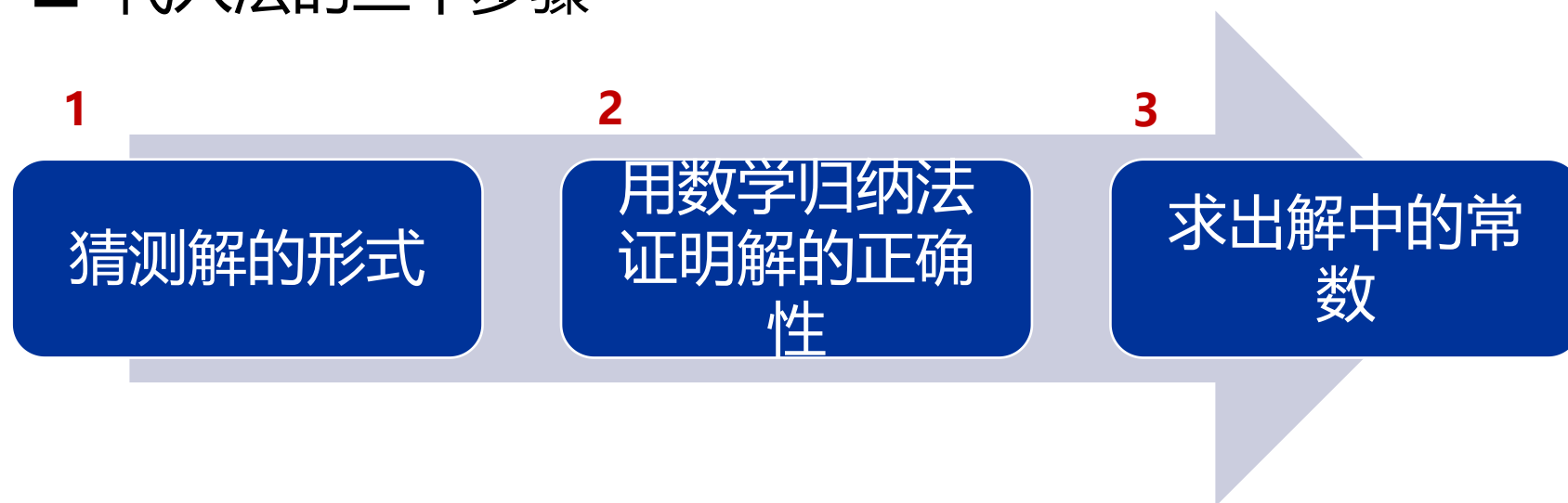


熟练掌握技巧

代入法（归纳法）的三个步骤

□ 代入法是最一般的求解递归式的方法

□ 代入法的三个步骤



代入法：例1

$$T(n) = \begin{cases} \Theta(1) & n = 1 \\ 4T(n/2) + n & n > 1 \end{cases}, \text{ 其中 } n = 2^h.$$

□ 猜测： $T(n) = O(n^3)$

□ 归纳假定： $T(n) \leq cn^3$ 。 c 是待定常数。

□ 不完全归纳法：

- 当 $n = 1$ 时，可取 c 足够大，使得 $T(1) = \Theta(1) \leq c$ 。
- 假设当 $n = k$ 时 $T(k) \leq ck^3$ ，那么

$$\begin{aligned} T(n) &= 4T\left(\frac{n}{2}\right) + n \leq 4c\left(\frac{n}{2}\right)^3 + n \\ &= c \cdot n^3 - \left(\frac{c}{2} \cdot n^3 - n\right) \\ &= \frac{c}{2} \cdot n^3 + n \end{aligned}$$

n^3 是上界成立

经验1：先证明较松的上界和下界，然后两边逐渐缩紧

代入法：例2

$$T(n) = \begin{cases} \Theta(1) & n = 1 \\ 4T(n/2) + n & n > 1 \end{cases}, \text{ 其中 } n = 2^h。$$

□ 已经证明 $T(n) = O(n^3)$

□ 进一步假设 $T(k) \leq ck^2$

$$T(n) = 4T\left(\frac{n}{2}\right) + n \leq 4c\left(\frac{n}{2}\right)^2 + n = c \cdot n^2 + n$$

无法证明 $T(n) \leq cn^2$ ，反而能证明 $T(n) \geq cn^2$!

代入法：例2

$$T(n) = \begin{cases} \Theta(1) & n = 1 \\ 4T(n/2) + n & n > 1 \end{cases}, \text{ 其中 } n = 2^h。$$

□ 已经证明 $T(n) = O(n^3)$

□ 进一步假设 $T(k) \geq ck^2$

$$T(n) = 4T\left(\frac{n}{2}\right) + n \geq 4c\left(\frac{n}{2}\right)^2 + n = c \cdot n^2 + n$$

证明出 $T(n) \geq cn^2$!

经验2：当归纳法证明失败时，在假设中减去一个低阶项

代入法：例2

□ **对策**：减去一个低阶的项，

□ **猜测**： $T(n) \leq c_1 n^2 - c_2 n = \Theta(n^2)$

□ **不完全归纳法**：

- 当 $n = 1$ 时，可取 c_1 足够大，使得 $T(1) = \Theta(1) \leq c_1 - c_2$ 。

- 假设当 $k < n$ 时 $T(k) \leq c_1 k^2 - c_2 k$ ，那么

$$T(n) = 4T\left(\frac{n}{2}\right) + n \leq 4 \left[c_1 \left(\frac{n}{2}\right)^2 - c_2 \left(\frac{n}{2}\right) \right] + n$$

$$= c_1 n^2 - c_2 n - (c_2 n - n)$$

$$\leq c_1 n^2 - c_2 n \quad (\forall c_2 > 1)$$

证明出 $T(n) \leq cn^2$!

证毕

代入法：例2

$$T(n) = \begin{cases} \Theta(1) & n = 1 \\ 4T(n/2) + n & n > 1 \end{cases}, \text{ 其中 } n = 2^h.$$



n^2 是渐进紧界

小结

□ 递归式分析方法比较

分析方法	优点	缺点
递归树法	直观形象	难以展开
主定理法	形式简洁	适用有限
代入法	适用广泛	难猜通解

作业

1. 用归纳法（代入法）证明

$$\begin{aligned} T(n) &\leq \begin{cases} 0 & \text{if } n = 1 \\ T\lfloor n/2 \rfloor + T\lceil n/2 \rceil + cn & \text{if } n > 1 \end{cases} \\ &\leq cn \lceil \log_2 n \rceil \end{aligned}$$

2. 应用master方法求解 $T(n) = 2T(n/2) + \Theta(n^{1/2})$
3. 展开递归树: $T(n) = T(2) + T(n-2) + cn$, 并做渐近分析
4. 展开 $T(n) = T(0.2n) + T(0.8n) + \Theta(n)$ 的递归树并计算递归树的深度和 $T(n)$ 的渐近值.
5. 计算算法A的时间复杂度, 算法满足递归关系式:
$$\begin{cases} f(n) = 2f(n-1) + 1 \\ f(0) = 1 \end{cases}$$

本章总结

□ (PPT04-09)递归法的概念及实例 □ (PPT40-72)解递归的三种方法

- 递归算法的基本概念
- 阶乘函数
- Fibonacci数列

□ (PPT11-38)递归法的解题思路

- 递归和迭代



汉诺塔问题

✓ $n=1$ & $n=2$ & $n>2$

汉诺塔问题不能用迭代求解可以用递归求解



考察重点



熟练掌握技巧



递归树(PPT40-53)

- 减法式递归
- 除法式递归
- 渐近值和深度
- 递归式



主方法(PPT55-63)

- 情形1&2&3



代入法(PPT65-71)

- 三步求解法

✓ 是求解递归式最一般的方法

附录：主方法证明

□ **首先证明** $T(n) = n^{\log_b a} T(1) + \sum_{k=1}^h a^{h-k} f(b^k)$

□ **不妨设** $n = b^h$, $T(1) = c_1$, 经过**迭代**得到

$$T(n) = aT(n/b) + f(n)$$

$$= a[aT(n/b^2) + f(n/b)] + f(n)$$

$$= a^2T(n/b^2) + af(n/b) + f(n)$$

$$= \dots$$

$$= a^kT(n/b^k) + a^{k-1}f(n/b^{k-1}) + \dots + af(n/b) + f(n)$$

$$= a^hT(1) + \sum_{j=0}^{h-1} a^j f(n/b^{h-j}) \quad // a^h = a^{\log_b n} = n^{\log_b a}$$

$$= c_1 n^{\log_b a} + \sum_{j=0}^{h-1} a^j f(b^{h-j})$$

证毕

$$// \sum_{j=0}^{h-1} a^j f(b^{h-j}) = \sum_{k=1}^h a^{h-k} f(b^k)$$

附录：主方法证明

已证明 $T(n) = n^{\log_b a} T(1) + \sum_{k=1}^h a^{h-k} f(b^k)$ 。

已知 $f(n) = O(n^{\log_b a - \varepsilon})$, **求证** $T(n) = \Theta(n^{\log_b a})$ 。

证明 由已知得

$$(b^k)^{\log_b a - \varepsilon} = (b^{\log_b a - \varepsilon})^k = (a/b^\varepsilon)^k = a^k / b^{\varepsilon k},$$

所以 $f(b^k) = O((b^k)^{\log_b a - \varepsilon}) = O(a^k / b^{\varepsilon k})$ 。

即存在常数 $c > 0$, 使得 $f(b^k) \leq c \cdot (a^k / b^{\varepsilon k})$ 。因此

$$\begin{aligned} \sum_{k=1}^h a^{h-k} f(b^k) &\leq c \cdot a^h \sum_{k=1}^h b^{-\varepsilon k} \leq c \cdot a^h \sum_{k=1}^{\infty} b^{-\varepsilon k} \leq c' \cdot a^h \\ &= O(n^{\log_b a}) \end{aligned}$$

即 $T(n) = n^{\log_b a} T(1) + \sum_{k=1}^h a^{h-k} f(b^k) = O(n^{\log_b a})$ 。

显然有 $T(n) = \Omega(n^{\log_b a})$, 所以 $T(n) = \Theta(n^{\log_b a})$ 。

附录：主方法证明

已证明 $T(n) = n^{\log_b a} T(1) + \sum_{k=1}^h a^{h-k} f(b^k)$ 。

已知 $f(n) = \Theta(n^{\log_b a})$, **求证** $T(n) = \Theta(n^{\log_b a} \log_b n)$ 。

证明 $n = b^h$, $n^{\log_b a} = a^h$, $n^\varepsilon = b^{\varepsilon h}$ 。因为
$$f(b^k) = \Theta\left((b^k)^{\log_b a}\right) = \Theta\left((b^{\log_b a})^k\right) = \Theta(a^k)$$

取充分大的常数 c , 使得 $f(b^i) = c \cdot (a^i)$, 我们有

$$\begin{aligned} T(n) &= n^{\log_b a} T(1) + \sum_{k=1}^h a^{h-k} f(b^k) \\ &= n^{\log_b a} T(1) + c \cdot a^h \cdot h \\ &= \Theta(n^{\log_b a} \log_b n) \end{aligned}$$

附录：主方法证明

已证明 $T(n) = n^{\log_b a} T(1) + \sum_{k=1}^h a^{h-k} f(b^k)$ 。

已知 $f(n) = \Omega(n^{\log_b a + \varepsilon})$ 并且 $af(n/b) \leq cf(n)$ 。

证明 $n = b^h$, $n^{\log_b a} = a^h$, $n^\varepsilon = b^{\varepsilon h}$, 由有

$$f(n) \geq \frac{a}{c} f\left(\frac{n}{b}\right) \geq \frac{a^2}{c^2} f\left(\frac{n}{b^2}\right) \geq \dots \geq \frac{a^j}{c^j} f\left(\frac{n}{b^j}\right)$$

所以

$$\sum_{k=1}^h a^{h-k} f(b^k) \leq \sum_{k=1}^h c^{h-k} f(n)$$

$$T(n) \leq n^{\log_b a} T(1) + f(n) \sum_{k=1}^h c^{h-k}$$

因为 $f(n) = \Omega(n^{\log_b a + \varepsilon})$, 量级比 $n^{\log_b a}$ 高, 所以 $T(n) = O(f(n))$. 又因为 $T(n) > f(n)$, 于是 $T(n) = \Theta(f(n))$ 。

14:35

Wednesday, October 16

26°C Good 27°C 75%

第3章：递归法

算法设计与分析



主讲教师：胡一涛